

CSCE 642 DEEP REINFORCEMENT LEARNING · MODULE 04 · LECTURE NOTES

Deep Q-Networks

What breaks when the table becomes a network, and why.

MODULE OUTCOMES

- State the deadly triad and explain each ingredient's role.
- Explain experience replay and target networks as fixes.
- Explain the main DQN improvements and which are substantial.
- Diagnose a diverging value-based agent.
- State what DQN is and is not suitable for.

1 The deadly triad

FUNCTION APPROXIMATION, BOOTSTRAPPING, AND OFF-POLICY LEARNING

Function approximation. A network generalises, so updating one state's value necessarily changes other states' values — including states where the change is wrong. A table has no such coupling.

Bootstrapping. The update target depends on the current estimate (Module 03 §1), **so an error in the estimate feeds into its own target** and can amplify rather than correct.

Off-policy learning. The states you update are not distributed as the policy being evaluated would visit them, **so the errors are weighted by the wrong distribution** and the states that would correct a growing error may be visited rarely or never.

Any two of the three is safe. All three together can diverge — provably, on examples with fewer than ten states (§1's code). **And DQN uses all three. So every stabilising mechanism in this module is damage control rather than refinement**, which is the right way to understand replay buffers and target networks: **they are not clever tricks that improve a sound algorithm; they are what makes an unsound combination usable.**

BAIRD'S COUNTEREXAMPLE

A seven-state MDP with a LINEAR value function whose features are deliberately chosen so that the states SHARE PARAMETERS.

Run off-policy TD on it, with a behaviour distribution that does not match the target policy's.

THE WEIGHTS DIVERGE TO INFINITY. Not oscillate; not converge slowly -- diverge, monotonically, without bound.

WHY: updating one state's value, through the shared parameters, changes the BOOTSTRAP TARGET of another state in a direction that increases its error. And the off-policy weighting means the correction never arrives, because the states that would supply it are visited rarely.

SEVEN STATES. A LINEAR approximator. No neural network, no stochasticity, no deep anything, no implementation bug.

SO WHEN A DEEP Q-NETWORK DIVERGES, IT IS NOT NECESSARILY A BUG IN YOUR CODE. It may be -- but the algorithm diverges on seven states too, and the mechanisms in section 2 exist because of this, not because deep networks are fiddly.

Knowing that a seven-state example diverges changes how you debug. You look for the triad before you look for a typo: is the target network actually being used? Is it updated too rarely or too often? Is the learning rate too high for the bootstrap depth? **Which is a different and more productive first hour than reading your replay buffer indexing.**

2 The two fixes

Mechanism	What it addresses	Side effect
Experience replay	Consecutive samples are highly correlated, which breaks the independence that stochastic gradient descent assumes (CSCE 669 Module 08); and it lets each sample be reused many times.	The buffer contains data from older, worse policies — which is the off-policy leg of the triad, deliberately accepted.
Target network	The bootstrap target moving as you update — see the callout.	Value propagates more slowly, so the update interval trades stability against speed.
Reward clipping to $[-1, 1]$	The loss scale varying wildly between tasks, which makes one learning rate work across games.	It destroys reward magnitude information — a 1-point and a 1000-point event become identical, which is a real loss and is rarely acknowledged.

Mechanism	What it addresses	Side effect
Huber loss	Large TD errors producing huge gradients that destabilise the network.	Slower learning precisely where the error is largest, which is sometimes where you need it fastest.
Frame stacking	Partial observability — a single frame does not reveal velocity, so the state is not Markov (Module 03 §1).	Four times the input size, and it only fixes the short-range case; longer memory needs a recurrent or attention-based architecture.

WHY A TARGET NETWORK WORKS

Without one, the target $r + \gamma \max_a Q(s', a)$ is computed with the *same* network you are updating — so each gradient step moves the target it was chasing, in a direction correlated with the step itself.

That is a positive feedback loop, and it is the bootstrapping leg of the triad at its worst — the estimate and its own target move together, so nothing anchors the scale.

Freeze a copy of the network, compute all targets from the frozen copy, and refresh the copy every few thousand steps. The target is now stationary between refreshes.

So each phase between refreshes is an ordinary supervised regression problem — fixed inputs, fixed targets, CSCE 636's entire toolkit applies — which is the right way to think about what DQN is doing. The cost is that value information propagates one target-refresh per step of distance, so the refresh interval is a genuine hyperparameter: too short and you reintroduce the feedback loop, too long and learning crawls.

3 The improvements, ranked

- **Double DQN.** Fixes a real, understood, systematic bias (Module 03 §4) for approximately one line of code — use the online network to select the action and the target network to evaluate it. **Take it unconditionally.**
- **n-step returns.** Cheap and reliably helpful, exactly as in the tabular case (Module 03 §3). Three to five steps.
- **Prioritised experience replay.** Sample transitions with large TD error more often, on the reasoning that they carry more information. **It helps, and it adds importance-weight bookkeeping that is easy to get subtly wrong** — the correction for the non-uniform sampling is required, and omitting it introduces a bias that is hard to notice.
- **Duelling architecture.** Split the network into a state-value stream and an action-advantage stream and recombine them, so the value of a state can be learned without distinguishing actions. **Modest, cheap, and uncontroversial.**
- **Distributional reinforcement learning (C51, QR-DQN).** Learn the full distribution of returns rather than its mean. A large and reliable gain — and the explanation for why is still debated, since the policy only uses the mean. The leading guesses are a better-conditioned learning signal and an implicit auxiliary task; neither is settled, and saying so is more useful than picking one.
- **Rainbow combines all of these and is substantially better than DQN** — and its ablation study shows prioritised replay and multi-step returns contribute most, which is worth knowing before implementing all six.

4 Diagnosis

Q-VALUES GROWING WITHOUT BOUND

the triad (section 1). Check the target network is actually being used and is being refreshed; reduce the learning rate; add double Q-learning; check the reward scale.

Q-VALUES PLAUSIBLE, EPISODE RETURN FLAT

exploration, or the reward itself. Plot state visitation (Module 02 section 4). If the rewarding region is never reached, it is exploration; if it is reached and nothing improves, it is not.

LEARNS, THEN COLLAPSES

the classic failure. Usually the replay buffer filling with data from a policy that has since changed, or the target refresh interval being too long relative to how fast the policy is moving.

WORKS ON ONE SEED AND NOT OTHERS

THIS IS NORMAL (Module 12), not a bug. Run five seeds before concluding anything about anything.

LOSS DECREASING, RETURN NOT IMPROVING

THE LOSS IS NOT THE OBJECTIVE. A TD loss can fall steadily while the policy gets worse, because the network is becoming self-consistent rather than correct. NEVER judge a reinforcement learning run by its loss curve. Judge it by episode return.

That last one is the most common mistake made by people arriving from supervised learning, where the training loss genuinely is a proxy for the thing you want. **Here it is not:** the TD loss measures self-consistency between the network and its own bootstrap targets, and **a network can become perfectly self-consistent around a wrong value function. The only reliable signal is the episode return**, and the only reliable comparison is across seeds (Module 12).

WHERE DQN BELONGS

Discrete action spaces, and small ones. The max over actions in the target is computed by enumeration, so a large discrete space is expensive and a continuous one is impossible (Module 07 exists for this reason).

Plenty of samples available. Tens of millions in practice for anything visually complex — Module 01 §2's figure, which means a fast simulator is a precondition.

And off-policy data worth reusing, which is **the actual advantage over the policy-gradient methods of Module 05**: a replay buffer means each environment step contributes to many gradient steps, which is where the sample-efficiency advantage comes from.

Not for continuous control (Module 07), **not where samples are scarce** (Module 08's model-based methods), **and not where you need a genuinely stochastic policy** — which Module 05's methods represent natively and a greedy value-based method does not. **Knowing the three exclusions is most of knowing when to reach for something else.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Mnih et al. — Human-level control through deep reinforcement learning (free)	The DQN paper. The replay and target-network motivation of §2 is stated here, and the engineering details matter.
Sutton & Barto — chapter 11 (free PDF)	The deadly triad and Baird's counterexample of §1 , developed carefully. This chapter is the one to read slowly.
Hessel et al. — Rainbow: Combining Improvements in Deep Reinforcement Learning (free)	§3's list with the ablation study that says which components actually contribute.
CleanRL — <code>dqn.py</code> and <code>dqn_atari.py</code> (free)	A readable single-file DQN with every implementation detail visible — and the details are where the performance is.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement Baird's counterexample** and plot the weights diverging.
- 2 **Remove one leg of the triad** (make it on-policy) and confirm it converges.
- 3 **Implement DQN on CartPole** from scratch, with replay and a target network.
- 4 **Ablate the replay buffer** and report what happens.
- 5 **Ablate the target network** and report what happens.
- 6 **Sweep the target refresh interval** and plot performance against it.
- 7 **Add double Q-learning** and measure the change in Q-value magnitudes.
- 8 **Add n-step returns** and report the improvement.
- 9 **Plot the TD loss and the episode return on the same figure** for a run that fails, and confirm they disagree.
- 10 **Run five seeds** of your best configuration and plot all five.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the three legs of the deadly triad and what each contributes.
- 2 What does Baird's counterexample establish, and with how many states?
- 3 How should that change your debugging?
- 4 What does experience replay address, and what is its side effect?
- 5 Explain precisely why a target network helps, and what it costs.
- 6 Which DQN improvements are cheap and reliable?
- 7 Which is largest and least understood?
- 8 Give five symptoms and their likely causes.
- 9 Why is the loss not the objective?
- 10 Name three situations where DQN is the wrong choice.